

ECE3411

Lecture 10

Digital Analog Converter

Sung Yeul Park

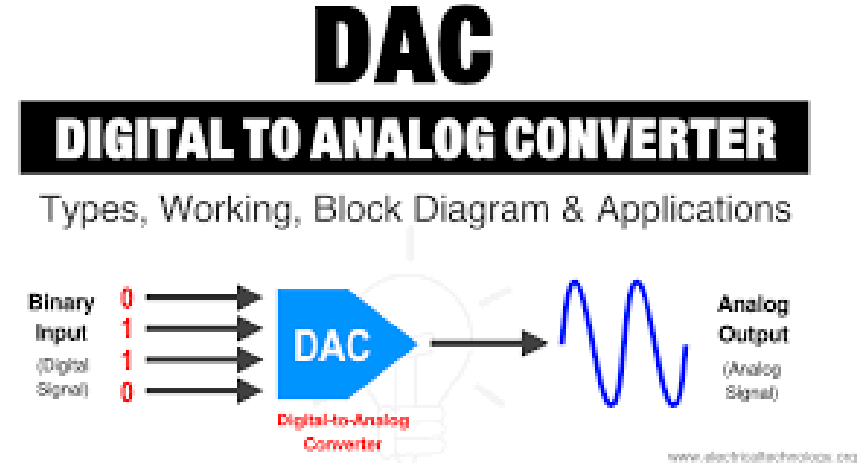
Department of Electrical & Computer Engineering

University of Connecticut

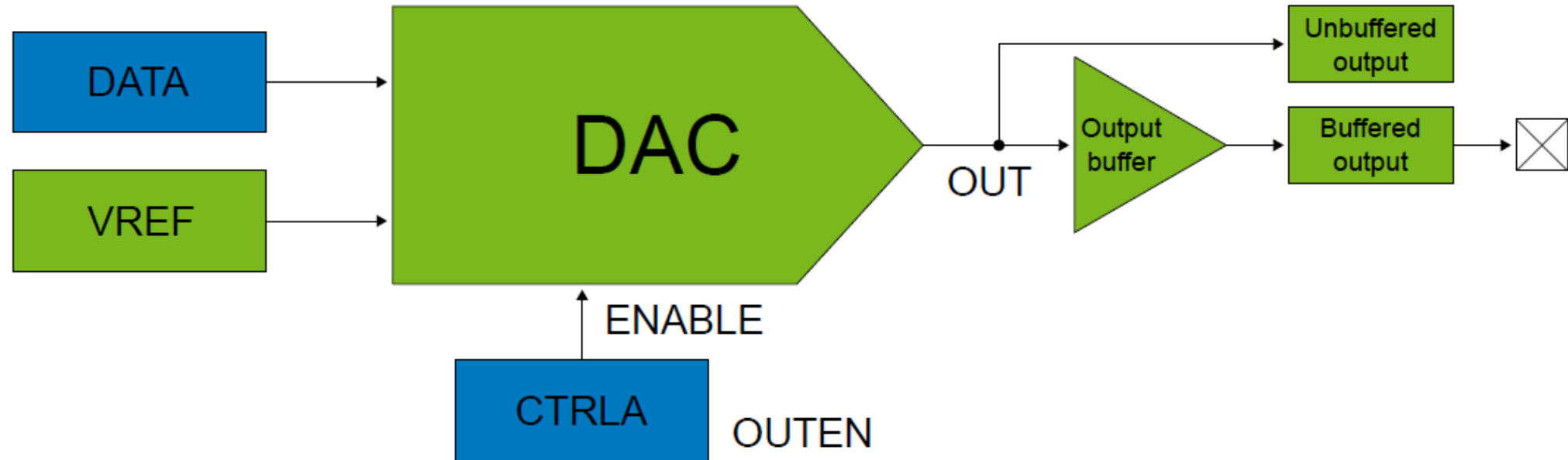
Email: sung_yeul.park@uconn.edu

Introduction

- **1. What is DAC?**
- **DAC (Digital-to-Analog Converter)** converts digital values (binary) into analog voltage.
- Common applications:
 - Audio signal generation
 - Analog control signals
 - Sensor simulation
- **2. DAC Features in AVR128DB48**
- 10-bit resolution
- Output range: 0V to VDD
- Buffered output on pin PD6 (DAC0 output)
- Can be triggered manually or by events (e.g., timer)



AVR128DB48 DAC



Register	Description
DACn.CTRLA	Enable DAC, select run mode
DACn.DATA	10-bit digital value to convert

DAC Control A Register

Bit	7	6	5	4	3	2	1	0
	RUNSTDBY	OUTEN						ENABLE
Access	R/W	R/W						R/W
Reset	0	0						0

Bit 7 – RUNSTDBY Run in Standby Mode

If this bit is written to '1', the DAC or the output buffer will not automatically be disabled when the device is entering Standby sleep mode.

Bit 6 – OUTEN Output Buffer Enable

Writing a '1' to this bit enables the output buffer and sends the OUT signal to a pin.

Bit 0 – ENABLE DAC Enable

Writing a '1' to this bit enables the DAC.

DAC Data Register

The DACn.DATAL and DACn.DATAH register pair represents the 10-bit value DACn.DATA in the following way:

- DACn.DATA[9:2] = DACn.DATAH (can be accessed at offset + 0x01)
- DACn.DATA[1:0] = DACn.DATAL (can be accessed at original offset)

The output will be updated after DACn.DATAH is written.

Bit	15	14	13	12	11	10	9	8
	DATA[9:2]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	DATA[1:0]							
Access	R/W	R/W						
Reset	0	0						

Bits 15:6 – DATA[9:0]

These bits contain the digital data that the DAC peripheral will convert to an analog value and send to the DAC output.

DACOREF

Bit	7	6	5	4	3	2	1	0
	ALWAYSON					REFSEL[2:0]		
Access	R/W					R/W	R/W	R/W
Reset	0					0	0	0

Bit 7 – ALWAYSON Reference Always On

This bit controls whether the DAC0 reference is always on or not.

Value	Description
0	The reference is automatically enabled when needed
1	The reference is always on

Bits 2:0 – REFSEL[2:0] Reference Select

This bit field controls the reference voltage level for DAC0.

Value	Name	Description
0x0	1V024	Internal 1.024V reference ⁽¹⁾
0x1	2V048	Internal 2.048V reference ⁽¹⁾
0x2	4V096	Internal 4.096V reference ⁽¹⁾
0x3	2V500	Internal 2.500V reference ⁽¹⁾
0x4	-	Reserved
0x5	VDD	VDD as reference
0x6	VREFA	External reference from the VREFA pin
0x7	-	Reserved

Note:

1. The values given for internal references are only typical. Refer to the *Electrical Characteristics* section for further details.

Sample Code for DAC

```
#include <avr/io.h>
#include <avr/pgmspace.h>
#include <util/delay.h>

// Sine wave lookup table(64 samples, 10-bit resolution)
const uint16_t sine_table[64] PROGMEM = {
    512, 562, 612, 660, 707, 752, 794, 833,
    868, 900, 927, 950, 968, 982, 991, 995,
    995, 991, 982, 968, 950, 927, 900, 868,
    833, 794, 752, 707, 660, 612, 562, 512,
    461, 411, 363, 316, 271, 229, 190, 155,
    123, 95, 72, 53, 39, 30, 25, 24,
    28, 37, 50, 68, 90, 116, 146, 180,
    218, 259, 303, 350, 399, 449, 500, 551
};

void DAC_init(void) {
    DAC0.CTRLA = DAC_ENABLE_bm | DAC_OUTEN_bm;

    // Set DAC reference to VDD (default, typically 3.3V)
    VREF.DAC0REF = VREF_REFSSEL_VDD_gc;
}
```

```
int main(void) {
    uint8_t index = 0;

    DAC_init();

    while(1) {
        // Read value from lookup table and output to DAC
        uint16_t sample = pgm_read_word(&sine_table[index]);

        DAC0.DATAL = sample & 0x03;    // write low 2 bits
        DAC0.DATAH = (sample >> 2) & 0xFF;

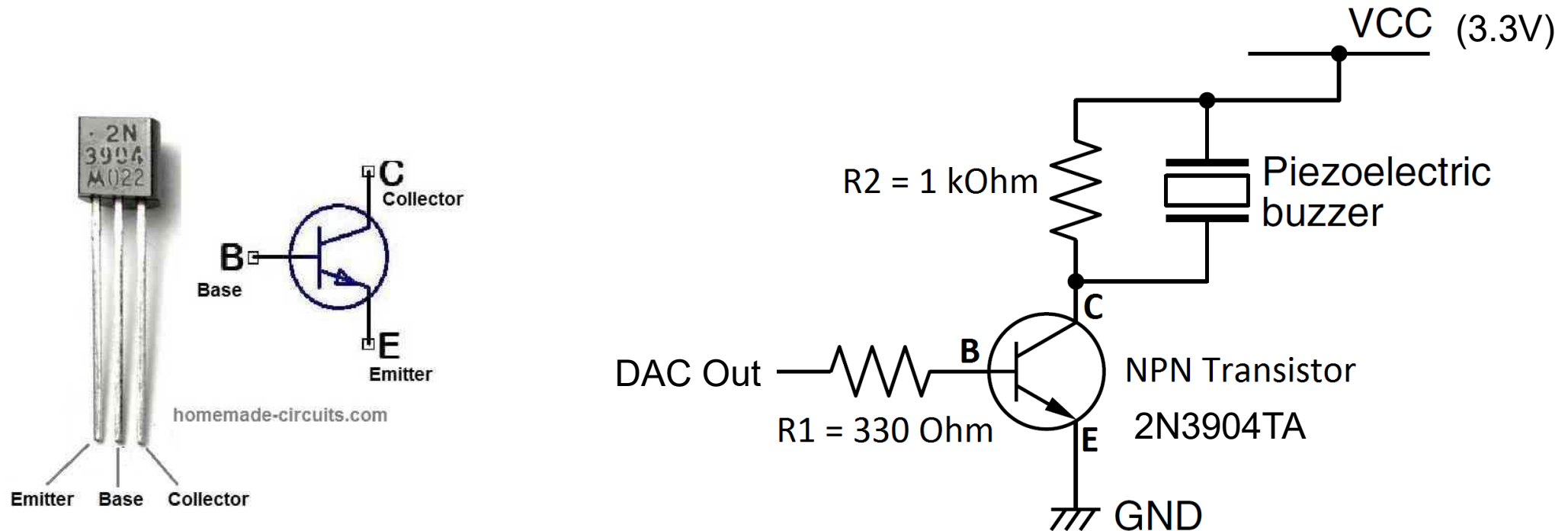
        // Move to next sample
        index++;
        if (index >= 64) {
            index = 0;
        }

        // Adjust delay for desired frequency
        _delay_us(5);
    }

    return 0;
}
```

Buzzer Circuit

- In this lab, we'll be using a buzzer that'll be driven by a DAC signal.
- Connect the buzzer according to the following schematic.



Lab Practice#9: Sine wave generator

- You will create a **Programmable Waveform Generator** that accepts commands via USART to control DAC output waveforms. You need to connect DAC output to the buzzer circuit.
- **System Requirements**
- Generate sine waveform (verify with oscilloscope)
- Type F or A to update values
 - Variable frequency control (10Hz to 1kHz)
 - Variable amplitude control (10% to 100% of reference voltage)
- Real-time status reporting via USART every 5 seconds
- Non-blocking operation using interrupts